MAE5920 · INTRODUÇÃO A DEEP LEARNING

# O que a CNN lê em uma resenha?

Classificação de sentimento e tokenização no IMDb

Nestor, Letícia e Gabriel

**Resumo**

Classificamos resenhas do IMDb em positivas e negativas com regressão logística sobre TF-IDF, uma CNN com embeddings aprendidos e encoders pré-treinados. O experimento original avalia os modelos no teste oficial. Ensaios internos com três sementes separam escolhas de otimizador, arquitetura e tokenização. Na comparação pareada, BPE reduziu tokens desconhecidos de 3,27% para zero e a acurácia média passou de 82,00% para 81,41%. Trocamos a posição de dois adjetivos para mudar a leitura de positiva para negativa. Os seis checkpoints trocaram de classe na direção oposta. Um controle com as mesmas janelas convolucionais produziu logits idênticos em cada checkpoint. Fora do IMDb, as CNNs obtiveram de 68% a 70% de acurácia média em frases de Amazon e Yelp.

## 1 Ordem e sentimento

Considere duas resenhas que só trocam os lugares de *terrible* e *wonderful*. Pela avaliação após *but*, a leitura adotada é positiva para A e negativa para B.

**A** *I thought this movie was terrible, but it was wonderful.*

**B** *I thought this movie was wonderful, but it was terrible.*

As duas sequências têm a mesma contagem de cada palavra. Um modelo que recebe apenas unigramas vê o mesmo vetor para A e B. Os bigramas mudam, portanto o TF-IDF de referência deste trabalho, que inclui bigramas, pode separá-las. A CNN também pode responder de modo diferente, pois aplica filtros a janelas ordenadas de embeddings. A questão empírica é se ela atribui o sentimento à oração que encerra o contraste.

Primeiro, medimos desempenho em resenhas reservadas e comparamos escolhas de treino e tokenização. Depois, usamos pesos congelados nos pares sintéticos para verificar sensibilidade à ordem e construir um limite exato da arquitetura. Avaliamos os mesmos checkpoints em outras fontes para examinar o que muda quando o texto deixa de ser uma resenha de filme.

Código, notebook executado e resultados por execução. Repositório do projeto.

## 2 Dados e protocolo de avaliação

O Large Movie Review Dataset contém 25.000 avaliações rotuladas no treino oficial e 25.000 no teste [1]. A tarefa usa o texto em inglês e dois rótulos, $y = 0$ para negativo e $y = 1$ para positivo. Decodificamos entidades HTML, removemos tags e normalizamos espaços. A identificação de duplicatas usa o hash do texto limpo com *casefold*.

### Papéis das amostras

| Papel | Original | Ensaios internos | Uso |
|---|---|---|---|
| Treino | 19.823 | 6.144 | Ajustar representação, vocabulário e pesos. |
| Validação | 2.478 | 1.536 | Escolher hiperparâmetros e época. |
| Calibração | 2.478 | – | Ajustar temperatura após a seleção original. |
| Teste/auditoria | 25.000 | 2.048 | Avaliar a configuração já escolhida. |

Tabela 1. O número 2.048 se refere à auditoria de cada extensão. CNN/encoders, palavras/BPE e MLP usam amostras internas distintas.

No preparo original, removemos do treino textos com hash presente no teste, duplicatas internas e conflitos de rótulo. A comparação de hashes usa os textos do teste para excluir duplicatas, com rótulos reservados à avaliação. Os registros distinguem 123 linhas do treino que coincidiam com o teste e 98 duplicatas internas do treino. O teste oficial conserva 201 ocorrências duplicadas. A auditoria detecta igualdade normalizada, de modo que paráfrases e possíveis exposições no pré-treinamento continuam como fontes de incerteza.

As extensões partem do treino limpo. CNN/encoders e palavras/BPE compartilham treino e validação e recebem prefixos de até 128 palavras. Cada rede pode usar até 256 tokens. A auditoria BPE exclui também os papéis da extensão sequencial. A entrada de cada auditoria ficou reservada até o congelamento dos pesos e das escolhas. Os resultados de teste oficial vêm da execução original. Manifesto original. Protocolo sequencial. Protocolo BPE.

### Dicionário resumido

| Campo | Significado |
|---|---|
| `id` | Identificador derivado da divisão oficial e da posição na fonte. |
| `label` | Classe binária usada na função de perda e na avaliação. |
| `clean_text` | Texto após decodificação de HTML e normalização de espaços. |
| `word_count` | Número de segmentos separados por espaço no texto limpo. |
| `text_sha256` | Impressão do texto normalizado usada para verificar duplicações. |

Tabela 2. O texto é a entrada do classificador. ID, rótulo e hash têm papéis de preparação ou avaliação. Dicionário completo.

### Métricas

Acurácia conta a proporção de classificações corretas. F1 macro calcula o F1 de cada classe e faz a média com pesos iguais. A entropia cruzada binária, $\mathrm{CE} = -n^{-1} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$, avalia o escore $p_i$ atribuído ao sentimento positivo. Ela aumenta quando a classe observada recebe probabilidade baixa. Na execução original, o cálculo salvo limita as probabilidades a $[10^{-7}, 1 - 10^{-7}]$. Os ensaios internos calculam CE diretamente dos logits.

# 3 Tokenização e arquitetura

## Duas unidades de entrada

**Palavras.** Convertemos o prefixo limpo para minúsculas. A expressão `[a-z]+(?:'[a-z]+)?|\d+` preserva palavras, contrações e números. O vocabulário contém os 19.998 itens mais frequentes no treino, além de `<pad>` (ID 0) e `<unk>` (ID 1). A frequência determina a ordem, com desempate lexical. A posição de cada ID é preservada na sequência. Palavras de negação como *not* seguem para o modelo.

**BPE.** Mantemos a normalização lexical e separamos palavras com `WhitespaceSplit`. O BPE de caracteres aprende uniões de pares frequentes com os 6.144 textos de treino, frequência mínima 2 e vocabulário de 20.000 itens. O marcador `</w>` identifica o fim de palavra. O tokenizer é congelado antes do treino supervisionado da CNN. O procedimento adapta BPE para unidades de texto [3].

| | |
|---|---|
| **Entrada** | *The pallbearer scene was unexpectedly moving.* |
| **Palavras** | `the <unk> scene was unexpectedly moving` |
| **BPE** | `the</w> p all bearer</w> scene</w> was</w> unexpectedly</w> moving</w>` |

Tabela 3. Saída dos tokenizadores congelados. `pallbearer` tem um ID desconhecido com palavras e três peças com BPE.

## Convolução sobre embeddings

Cada ID acessa um vetor de 128 dimensões aprendido com a tarefa. O filtro de largura $k$ lê $k$ tokens vizinhos e usa os mesmos pesos em todas as posições. Aplicamos três grupos paralelos de convoluções de larguras 3, 4 e 5, com cem filtros por largura. A ReLU deixa respostas positivas passarem. O máximo global seleciona a maior ativação por filtro. A topologia segue a família de CNNs para classificação de sentenças [2, 4].

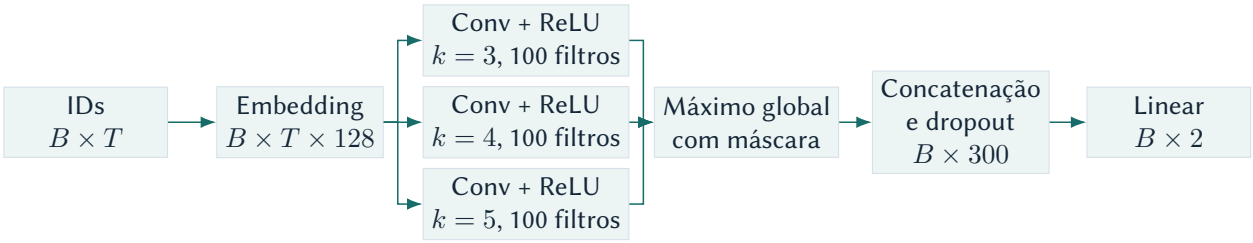



Figura 1. TextCNN do projeto. $B$ é o tamanho do minibatch e $T$ é o comprimento após padding. A camada linear produz logits negativo e positivo.

Se $e_t$ é o embedding na posição $t$, o filtro $f$ resume a entrada por

$$m_f = \max_{t \in \mathcal{V}_k} \mathrm{ReLU}(\langle W_f, [e_t, \ldots, e_{t+k-1}] \rangle + b_f) \,.$$

$\mathcal{V}_k$ exclui as janelas que avançam sobre padding em entradas com pelo menos $k$ tokens. Para sequências mais curtas, a implementação mantém a primeira janela com padding. A concatenação dos 300 máximos passa por dropout de 0,5 no treino e uma camada $\mathrm{Linear}(300, 2)$.

Na CNN principal, $20.000 \times 128 = 2.560.000$ parâmetros pertencem aos embeddings. As convoluções somam $100 \sum_{k \in \{3,4,5\}} (128k + 1) = 153.900$. A saída soma $2(300 + 1) = 602$, totalizando **2.714.502 parâmetros**. A mesma largura em tokens pode abranger menos palavras quando BPE fragmenta uma palavra. Implementação.

# 4 Treinamento e escolha dos hiperparâmetros

O forward produz dois logits. A entropia cruzada compara esses logits com o rótulo, e o *backward* calcula gradientes para embeddings, filtros e saída. Limitamos a norma global a 1 antes do passo do otimizador. Os embeddings começam com distribuição normal de desvio 0,1 e linha de padding zero. Os filtros usam inicialização de He, e a saída usa Xavier. A escolha de inicialização, taxa e regularização liga o experimento aos capítulos de otimização e generalização de Prince [4].

### Grade da CNN sequencial

| Otimizador | Taxas testadas | Taxa escolhida |
|---|---|---|
| SGD | 0,001 0,03 0,3 | 0,3 |
| SGD com momentum 0,9 | 0,001 0,01 0,1 | 0,1 |
| Adam | 0,0003 0,001 0,003 | 0,001 |

Tabela 4. Três sementes por célula. CNN base com janelas 3/4/5 e cem filtros por largura, dropout 0,5, batch 64 e orçamento de 12 épocas.

Dentro de cada semente, comparamos os otimizadores com as mesmas matrizes iniciais e a mesma ordem de minibatches. Escolhemos a época de menor CE de validação em cada execução. Em seguida, escolhemos a taxa com menor média dessas perdas nas sementes 17, 29 e 43. A época zero mostra a inicialização e fica fora da seleção.

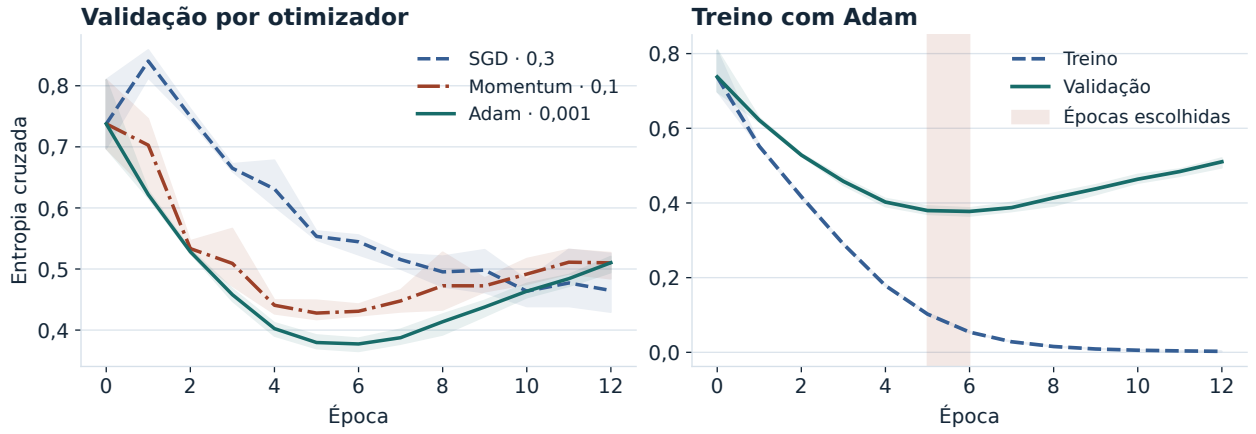



Figura 2. Médias de três sementes. As áreas mostram mínimo e máximo. À esquerda, cada otimizador usa sua taxa selecionada. À direita, as épocas 5 e 6 contêm os checkpoints escolhidos para Adam.

### Capacidade e regularização

Além dos 27 ajustes de otimizador e taxa, fizemos nove treinos para janelas 3 e 5 isoladas e janelas 3/4/5 com 32 filtros por largura. Seis treinos adicionais variaram dropout entre 0 e 0,8. As variantes reutilizam a taxa de Adam da CNN base. São **42 treinos de CNN** no estudo sequencial. Na auditoria, a acurácia foi 81,27% com $k = 3$, 80,94% com $k = 5$ e 81,95% com $k = 3, 4, 5$. A última variante combina três larguras e 300 filtros no total. As duas primeiras usam 100 filtros. Com 32 filtros por largura, a CNN chegou a 81,64%. O dropout de 0,8 resultou em 80,44%.

### Passo da janela na inferência

A largura $k$ define quantos tokens o filtro lê. O passo $s$ define quanto a janela avança. Mantivemos os pesos das três CNNs por palavras e aplicamos passos 1, 2 e 3 na auditoria da Seção 6. A acurácia média foi 82,00%, 81,45% e 80,97%, respectivamente. O ensaio mede a perda de janelas em redes treinadas com passo 1. Grade e resultados por passo.

# 5 Resultados de classificação

### Treino, validação e teste oficial

O baseline ajusta TF-IDF de unigramas e bigramas no treino e seleciona a regularização da regressão logística na validação. Ele recebe o texto completo. A CNN original usa até 400 tokens. O BERT recebe até 192 WordPieces e passa por ajuste dos pesos [5]. Esses comprimentos e o pré-treinamento fazem parte de cada receita.

| Divisão | Modelo | Acurácia | F1 macro | CE bruta |
|---|---|---|---|---|
| Treino | TF-IDF + logística | 99,72% | 0,9972 | 0,0973 |
| Treino | CNN | 94,95% | 0,9494 | 0,1567 |
| Treino | BERT | 96,32% | 0,9632 | 0,1138 |
| Validação | TF-IDF + logística | 91,53% | 0,9152 | 0,2255 |
| Validação | CNN | 87,97% | 0,8796 | 0,2877 |
| Validação | BERT | 91,28% | 0,9128 | 0,2404 |
| Teste | TF-IDF + logística | 90,14% | 0,9014 | 0,2531 |
| Teste | CNN | 86,78% | 0,8678 | 0,3040 |
| Teste | BERT | 90,94% | 0,9094 | 0,2433 |

Tabela 5. Probabilidades antes da calibração. Treino com 19.823 resenhas, validação com 2.478 e teste oficial com 25.000. A Tabela 1 descreve a divisão de calibração. CSV das divisões.

No teste oficial, o baseline tem 90,14% de acurácia, a CNN 86,78% e BERT 90,94%. A análise por comprimento e a calibração estão na Seção 7.

### Resenhas em que as duas receitas discordam

No teste oficial, só o TF-IDF acertou 1.715 resenhas e só a CNN acertou 875. A vantagem do TF-IDF é mais frequente nos textos que excedem 400 tokens. Em uma resenha negativa de 780 tokens, ambos dão positivo ao prefixo e negativo ao texto inteiro. Em outra resenha, positiva e com 61 tokens, *part is brilliant* está entre as janelas que mais empurram a CNN para a classe positiva. Casos e contribuições à decisão.

### Auditoria interna do estudo sequencial

Comparamos CNN e encoders nos mesmos prefixos de até 128 palavras, em 2.048 resenhas reservadas. Os encoders produzem estados contextuais. Usamos a média dos estados finais com máscara de padding e tokens especiais, seguida de uma cabeça linear de dois logits. Em um regime, apenas a cabeça é ajustada. No outro, os pesos do encoder também são atualizados.

| Modelo | Acurácia | F1 macro | CE |
|---|---|---|---|
| TF-IDF + logística | 83,30% | 0,8330 | 0,3849 |
| CNN + Adam | 81,95% | 0,8194 | 0,4094 |
| BERT-base ajuste completo | 88,31% | 0,8831 | 0,3027 |
| DistilBERT ajuste completo | 86,80% | 0,8678 | 0,3382 |
| ModernBERT-base ajuste completo | 91,24% | 0,9124 | 0,2458 |

Tabela 6. Baseline determinístico e médias das três sementes nas redes. Encoders na tabela com ajuste completo. Os escores dessa auditoria usam o softmax sem calibração de temperatura.

Nos encoders congelados, BERT, DistilBERT e ModernBERT alcançaram 84,52%, 83,35% e 81,30%. Cada receita tem seu orçamento e sua taxa. Métricas, sementes e configurações.

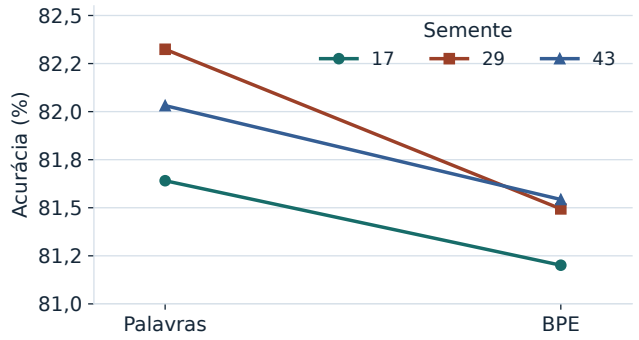
# 6 Palavras, subpalavras e ordem

## Comparação pareada de tokenizadores

Mantivemos arquitetura, dimensões, dropout, inicialização numérica, minibatches e orçamento. Treinamos três CNNs com BPE e reutilizamos três checkpoints por palavras. Ambas as receitas usam Adam a 0,001, escolhido antes no estudo por palavras. Cada execução seleciona a época na validação. O pareamento controla os pesos iniciais como matrizes numéricas. As linhas de embedding correspondem a unidades distintas em cada vocabulário.

| Tokenizer | Acurácia | F1 macro | CE | UNK | Tokens médios |
|---|---|---|---|---|---|
| Palavras | 82,00% | 0,8200 | 0,3964 | 3,27% | 121,3 |
| BPE 20 mil | 81,41% | 0,8140 | 0,4052 | 0,00% | 129,1 |

Tabela 7. Nova auditoria interna, com 2.048 resenhas. Métricas médias de três sementes. UNK conta a proporção de tokens desconhecidos.




A diferença média é de $-0,59$ ponto percentual para BPE. A cobertura melhorou e a sequência ficou mais longa. Um filtro de cinco tokens pode cobrir menos palavras, o que muda o contexto que ele observa. Esta comparação estima o desempenho sob a taxa herdada e as larguras fixas.

Figura 3. Cada linha une a mesma semente sob as duas tokenizações. Eixo vertical ampliado. Métricas por execução.

## Pares sintéticos com a mesma contagem de palavras

Fixamos quatro pares e suas leituras antes da inferência em `seminario/casos.json`. Dentro de cada par, mantivemos a contagem de cada palavra e permutamos duas partes do texto. Os mesmos seis checkpoints receberam A e B. Medimos se a classe mudou e se a mudança acompanhou a leitura planejada. Também verificamos as janelas de 3, 4 e 5 tokens de cada entrada.

| | | Palavras | | BPE | | |
|---|---|---|---|---|---|---|
| Par | Leitura A / B | A | B | A | B | Trocas |
| Contraste depois de but | Pos. / Neg. | 1,0% | 89,9% | 1,2% | 83,7% | 3/3 e 3/3 |
| Escopo de not | Pos. / Neg. | 48,4% | 10,3% | 53,1% | 15,5% | 2/3 e 1/3 |
| Sentimento por aspecto | Mista / Mista | 29,9% | 0,3% | 20,0% | 0,4% | 0/3 e 0/3 |
| Troca de blocos com as mesmas janelas | Mista / Mista | 88,4% | 88,4% | 73,6% | 73,6% | 0/3 e 0/3 |

Tabela 8. Escores positivos médios sem calibração. A última coluna conta as trocas de classe entre A e B nas três sementes de palavras e nas três de BPE. As predições e janelas estão em `dados/pares_resultados.json`.

Em *The movie is good, not bad*, a troca de *good* e *bad* reduz o escore positivo médio nos dois tokenizadores. Na semente 17, a própria frase A recebe 53,7% com palavras e 47,5% com BPE. O limiar 0,5 dá classes diferentes para o mesmo texto nas duas receitas treinadas. A Seção 8 compara o erro no contraste com o caso em que preservamos as janelas e a rede devolveu os mesmos logits.

# 7 Calibração, comprimento e custo

## Confiança no teste oficial

Se $z$ é a margem entre os logits positivo e negativo, ajustamos uma temperatura $T > 0$ por CE na amostra de calibração. A probabilidade passa de $\sigma(z)$ a $\sigma(z/T)$. Como a temperatura preserva o sinal da margem, o limiar 0,5 preserva acurácia e F1 macro.

| Modelo | Acurácia | F1 macro | CE calibrada |
|---|---|---|---|
| TF-IDF + logística | 90,14% | 0,9014 | 0,2396 |
| CNN | 86,78% | 0,8678 | 0,3037 |
| BERT | 90,94% | 0,9094 | 0,2278 |

Tabela 9. Teste oficial, $n = 25.000$, com temperatura escolhida em 2.478 resenhas reservadas. Compare a CE bruta com a Tabela 5.

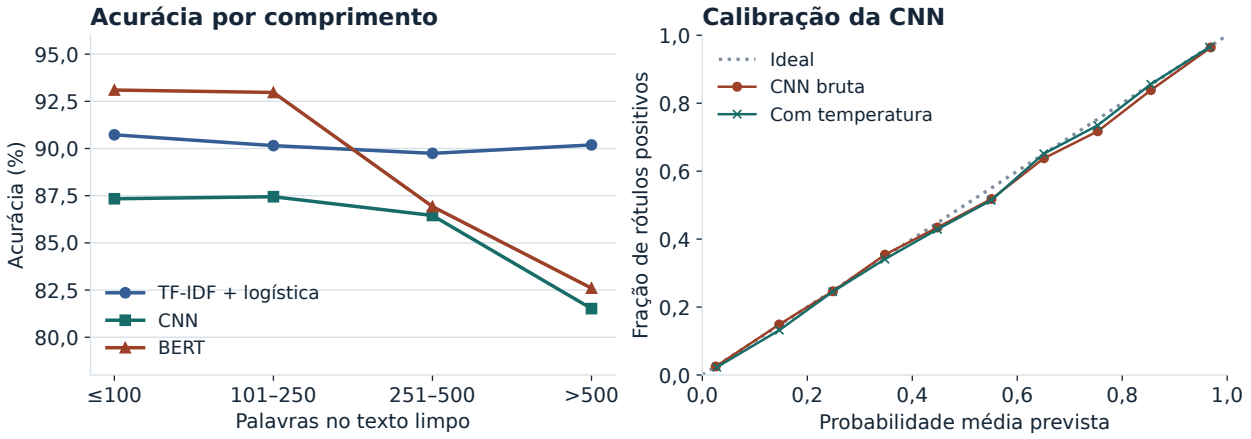



Figura 4. À esquerda, as quatro faixas de comprimento contêm 3.246, 14.624, 5.285 e 1.845 resenhas, respectivamente. À direita, médias de probabilidade e frequência positiva em dez faixas.

A temperatura teve efeito pequeno na CNN, com CE de 0,3040 para 0,3037. No comprimento, sua acurácia foi de 87,34% até cem palavras e 81,52% acima de 500. O BERT também cai na faixa longa, enquanto o TF-IDF com texto completo varia menos. Comprimento, conteúdo e recorte mudam juntos nessa análise descritiva. Métricas por comprimento.

## Custo computacional da execução original

| Modelo | Dispositivo | Ajuste (s) | Inferência (s) |
|---|---|---|---|
| TF-IDF + logística | CPU do Apple M1 | 13,1 | 0,15 |
| CNN | Apple M1 (MPS) | 139,6 | 20,72 |
| BERT | NVIDIA GB10 (CUDA) | 446,4 | 149,91 |

Tabela 10. Tempos registrados para as receitas originais. A inferência soma as quatro divisões. O ajuste do baseline inclui TF-IDF e a seleção de regularização. As medições usam dispositivos e orçamentos diferentes.

## Ensaios complementares

**MLPs.** O suplemento reúne 51 treinos sobre otimização, profundidade e regularização. O double descent aparece na regressão sintética com 880 condições. Guia de mecanismos.

**Qwen3-8B como modelo de linguagem.** Usamos o modelo pré-treinado com os pesos congelados [6]. Na classificação, comparamos os logits dos tokens `positive` e `negative`. Na geração, o modelo recebeu uma resenha ou um enredo e escreveu, token a token, um resumo factual (FACT) e uma opinião (REVIEW). Nesse segundo uso, mantivemos o prompt e testamos como um deslocamento nas ativações mudava o texto. Protocolo de geração.

# 8 Quando o modelo de aprendizagem profunda falha?

### A CNN reage à troca e inverte o sentido do contraste

A termina em *wonderful* e tem leitura positiva. B termina em *terrible* e tem leitura negativa. Todos os seis checkpoints deram classe negativa a A e positiva a B. Da primeira para a segunda frase, o escore positivo médio foi de 1,0% a 89,9% com palavras e de 1,2% a 83,7% com BPE.

### O máximo global apaga a posição entre janelas

Separamos *I loved the acting* de *I hated the ending* com uma frase repetida no início, no meio e no fim. O separador faz com que cada janela de até cinco tokens veja no máximo um dos trechos. Ao trocá-los, verificamos que os multiconjuntos de janelas de IDs para $k = 3, 4, 5$ permaneceram iguais nas duas tokenizações.

Janelas iguais geram as mesmas ativações para cada filtro. O máximo global retorna o mesmo valor para A e B, mesmo que a posição desse valor mude. Os vetores enviados à camada linear coincidem. A diferença máxima de logit foi **zero** nos seis checkpoints.

### O sentimento fora do IMDb

Aplicamos os mesmos pesos, sem reajuste, a 1.000 frases de Amazon e 1.000 de Yelp na coleção da UCI [7]. Cada fonte contém 500 rótulos positivos e 500 negativos. O gênero e o comprimento mudam juntos em relação ao IMDb.

| | Palavras | | | BPE | | |
|---|---|---|---|---|---|---|
| **Fonte** | **Acurácia** | **F1 macro** | **CE** | **Acurácia** | **F1 macro** | **CE** |
| Amazon | 68,10% | 0,6664 | 0,6372 | 69,97% | 0,6920 | 0,5900 |
| Yelp | 69,43% | 0,6828 | 0,6091 | 69,67% | 0,6911 | 0,5946 |

Tabela 11. Médias de três sementes em frases selecionadas por polaridade clara. Resultados por semente.

Em *The Room*, "No one" contesta o elogio entre aspas, mas os seis checkpoints classificaram o comentário como positivo. Os dois comentários de *Baby* também receberam classe positiva. Em *Friday*, "AI could never do this" recebeu classe negativa nos seis checkpoints. Cinco casos sem rótulos de referência.

### Tom e fatos na geração do Qwen

No bloco 28, aplicamos $h' = h + \alpha s v$ após o marcador REVIEW. A direção $v$ é a diferença normalizada entre as médias de ativações para dois tons em 128 enredos. Com escala $s$ fixa, usamos $\alpha = -2, 0, +2$. O próprio Qwen, como juiz automático, reconheceu os dois tons-alvo em 55 de 64 fontes. Os tokens de FACT coincidiram. Em um caso, a opinião positiva acrescentou um cenário de cidade pequena ausente da fonte. Leitura dos alertas factuais.

## Referências


[1] Maas, A. L. et al. *Learning Word Vectors for Sentiment Analysis.* ACL, 2011. ACL Anthology.

[2] Kim, Y. *Convolutional Neural Networks for Sentence Classification.* EMNLP, 2014. ACL Anthology.

[3] Sennrich, R., Haddow, B. e Birch, A. *Neural Machine Translation of Rare Words with Subword Units.* ACL, 2016. ACL Anthology.

[4] Prince, S. J. D. *Understanding Deep Learning.* MIT Press, 2023. Caps. 6, 8, 9 e 10. Livro e materiais.

[5] Devlin, J. et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL, 2019. ACL Anthology.

[6] Yang, A. et al. *Qwen3 Technical Report*, 2025. arXiv:2505.09388.

[7] Kotzias, D. et al. *From Group to Individual Labels Using Deep Features.* KDD, 2015. Artigo. Coleção UCI.